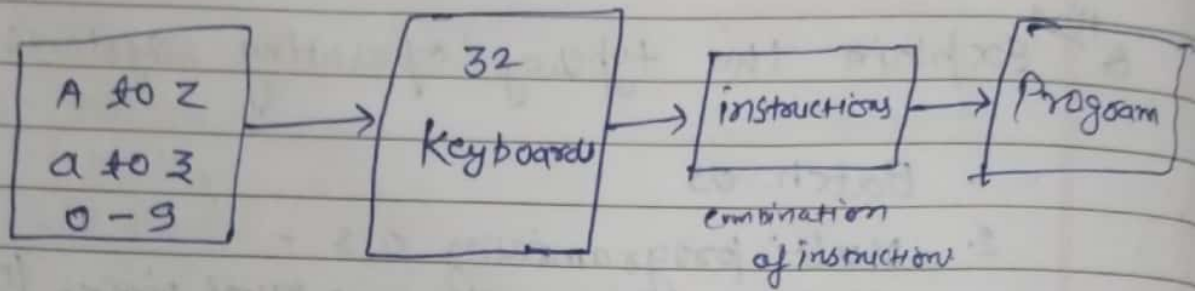
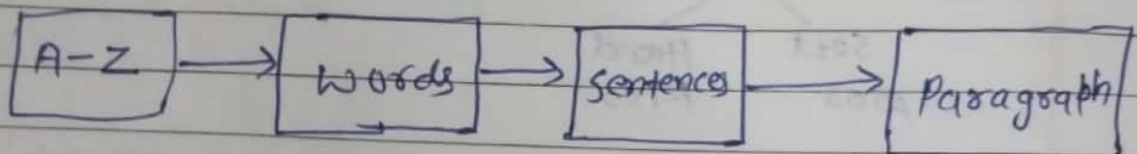


C- language : → 1970 → Dennis Ritchie



E- language



C- language : -

void main () → Starting of a Program (all small)
 #include <stdio.h> } Header files
 #include <conio.h> } or Dictionaries
 ; - - - semi column,

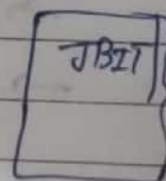
printf → for displaying
 scanf → for storing or scanning
 clrscr → clear screen
 getch → get character

display - JBIT

Sample :-

```

#include <stdio.h>
#include <conio.h>
void main ( )
{
  printf ("JBIT");
  getch ( );
}
  
```

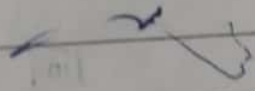


Sample Program

```
#include <stdio.h>
#include <conio.h>
void main ( )
{ clrscr ( );
```

Jbit
Fcb
Wajahat

```
→ printf ("Jbit\n") ;
→ printf ("Fcb\n") ;
→ printf ("Wajahat");
  getch ( );
}
```



display

(This is sec B CSE)

Sample-3

```
#include <stdio.h>
#include <conio.h>
void main ( )
{ clrscr ( );
  printf ("This is sec B CSE");
  getch ( );
}
```

Sample Program

```
* #include <stdio.h>
#include <conio.h>
void main()
{
    float b;
    → b = 20.7;
    printf ("%f", b);
    getch();
}
```

Note-

float → decimal
↳ %f

Int → Nondecimal/integer
↳ %d

↪ closes after float or int

Write a C-program to print addition of two numbers if the numbers are 10 & 20.

```
* #include <stdio.h>
#include <conio.h>
void main()
{
    Int a, b, c;
    a = 10;
    b = 20;
    c = a + b;
    printf ("%d", c);
    getch();
}
```

10	20	30
a	b	c

Write a C-program to print multiplication of two numbers if the numbers are 5.2 & 2.2.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    float a, b, c;
    a = 5.2;
    b = 2.2;
```

```

c = a * b;
printf ("%f", c);
getch();
}

```

Write C program to print addition of two numbers when numbers are entered in run time.

```

* #include <stdio.h>
#include <conio.h>
void main ()
{
    int x, y, z;
    clrscr();
    printf ("Enter value of x");
    scanf ("%d", &x);
    printf ("Enter value of y");
    scanf ("%d", &y);
    z = x + y;
    printf ("%d", z);
    getch();
}

```

3	6	9
x	y	z

Enter value of x
3
Enter value of y
6
9

* Write a c program to print Area of a rectangle :-

```
#include <stdio.h>
#include <conio.h>
void main ( )
{
    int l, b, a;
    clrscr ();
    printf ("enter length of a rectangle ");
    scanf ("%d", &l);
    printf ("enter breadth of a rectangle");
    scanf ("%d", &b);
    a = l * b;
    printf ("The area of rectangle is %d", a);
    getch ();
}
```

Write a c program to print perimeter of a rectangle.

```
#include <stdio.h>
#include <conio.h>
void main ( )
{
    int l, b, p;
    clrscr ();
    printf ("enter a length of rectangle ");
    scanf ("%d", &l);
    printf ("enter a breadth of rectangle ");
    scanf ("%d", &b);
    p = 2 * (l + b);
    printf ("perimeter of rectangle is %d", p);
    getch ();
}
```

* Write a C program to print Area of square.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int s, a;
    clrscr();
    printf("enter a value of side");
    scanf("%d", &s);
    a = s2;
    printf("%d", a);
    getch();
}
```

* Write a C program to print Area of right angle triangle.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int b, h, a;
    printf("enter a base of triangle b");
    scanf("%d", &b);
    printf("enter a height of triangle h");
    scanf("%d", &h);
    a =  $\frac{1}{2} \times b \times h$ ;
    printf("Area of right angle A is %d", a);
    getch();
}
```

Write a C program to print perimeter of a square.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main( )
```

```
{
```

```
    int s, p;
```

```
    clrscr();
```

```
    printf("enter the value of side");
```

```
    scanf("%d", &s);
```

```
    p = 4s
```

```
    printf("perimeter of square %d", p);
```

```
    getch();
```

```
}
```

06/09/23

Write a C-program to print addition and average of two numbers

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main( )
```

```
{
```

```
    int a, b, Sum, Avg;
```

```
    clrscr();
```

```
    → printf("enter value of a");
```

```
    scanf("%d", &a)
```

```
    printf("enter value of b");
```

```
    scanf("%d", &b);
```

```
    Sum = a + b;
```

```
    printf("the sum of 2 no's is %d", sum);
```

```
    Avg = sum/2;
```

```
    printf("The avg of 2 no's is %d", Avg);
```

```
    getch();
```

```
}
```

Write a C-program to print simple interest.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int p, r, t, s.I;
    clrscr();
    printf("enter the value of p");
    scanf("%d", &p);
    printf("enter the value of r");
    scanf("%d", &r);
    printf("enter the value of t");
    scanf("%d", &t);
    s.I =  $\frac{p \times r \times t}{100}$ ;
    printf("The value of s.I : %d", s.I);
    getch();
}
```

V.V.I

write a C-program to print swape of two numbers with using third variable.

```
#include <stdio.h>
#include <conio.h>
{
    int a, b, c;
    clrscr();
    printf("enter value of a, & b");
    scanf("%d %d", &a, &b);
```

```

c = a;
a = b;
b = c;
printf("The value of a & b is %d %d", a, b);
getch();
}

```

V.V.I.

Write a C-program to print swap of two numbers without using third variable.

```

#include <stdio.h>
#include <conio.h>
void main
{
    int a, b;
    clrscr();
    printf("enter value of a, & b");
    scanf("%d %d", &a, &b);

    a = a + b;
    b = a - b;
    a = a - b;

    printf("The value of a & b is %d %d", a, b);
    getch();
}

```

Conditional —, Write a C-program to print whether a number is +ve or -ve. (Format specified %d)

~~#include~~

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a ;
    clrscr ();
    printf ("enter any number ");
    scanf ("%d", &a);
    if (a >= 0)
    {
        printf ("No is +ve");
    }
    else
    {
        printf ("No is -ve");
    }
    getch ();
}
```

Write a C-program to print whether a number is even or odd.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int n;
```

```
clrscr();  
printf("enter any no.");  
scanf("%d", &n);
```

```
if (n%2 == 0)  
{  
    printf("even no!");  
}  
else  
    printf("odd no!");  
}  
getch;  
}
```

Write a C-program to print whether a year is leap or not.

Leap year or Not

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main ( )
```

```
{
```

```
    int year;
```

```
    printf ("enter any year");
```

```
    scanf ("%d", &year)
```

```
    if ((year % 400 == 0) || ((year % 4 == 0) & (year % 100 != 0)))
```

```
    {
```

```
        printf ("leap year");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf ("Not leap year");
```

```
    }
```

```
    getch();
```

```
}
```

Find the

Write a C-program to print whether the largest number among the two numbers.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main ( )
```

```
    int x, y;
```

```
    clrscr();
```

```
printf ("enter a number of x");  
scanf ("%d", &a);  
printf ("enter a number of y");  
scanf ("%d", &b);  
if (a > b)  
{ printf ("greater no a");  
}  
  
else  
{ printf ("not greater");  
}  
  
getch();  
}
```

Write a C-program to print largest no. among three numbers.

```
#include <stdio.h>
#include <conio.h>
{
    int a,b,c;
    clrscr();
    printf("enter value of a,b,c");
    scanf("%d %d %d", &a, &b, &c);
```

```
    if (a > b)
    {
        if (a > c)
        {
            printf("a is largest");
        }
```

```
    else
    {
        printf("c is largest");
    }
```

```
    }
    else
    {
        if (b > c)
        {
            printf("b is largest");
        }
```

```
    else
    {
        printf("c is largest");
    }
    getch();
}
```

loops.

↳ repeated process

for
while
do-while

Syntax of for loop

```
for (initialization; condition; increment)
{
    statement;
}
```

Sample Program of for loop;

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i;
    clrscr();
    for (i=1, i<5; i++)
    {
        printf("secB");
    }
    getch();
}
```

Write a c-program to print first ten natural Number.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i;
    clrscr();
```

```

for (i = 1, i <= 10; i++)
{
    printf ("%d", i);
}
getch();
}

```

write a c-program to print whole numbers first 10 whole number

~~## void main~~

```

#include <stdio.h>
#include <conio.h>
void main ()
{
    int int i;
    clrscr ();
    for (i = 0, i <= 9; i++)
    {
        printf ("%d", i);
    }
    getch ();
}

```

write a c-program to print n natural numbers.

```

#include <stdio.h>
#include <conio.h>
void main ()
{
    int i, n;
}

```

```

clear ()
printf ("enter value of n");
scanf ("%d", &n);
for (i=1; i<n; i++)
{
    printf ("%d", i);
}
getch ();
}

```

Write a C-program to print n whole number.

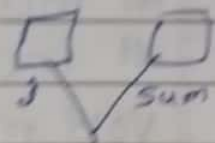
```

#include <stdio.h>
#include <conio.h>
void main ()
{
    int i, n;
    clear
    printf ("enter the value of n");
    scanf ("%d", &n);
    for (i=0; i<n; i++)
    {
        printf ("%d", i);
    }
    getch ();
}

```

Write a C-program to print sum of first ten natural numbers

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i, sum = 0;
    clrscr();
    for (i = 1, i <= 10; i++)
    {
        sum = sum + i;
    }
    printf("%d", sum);
    getch();
}
```



Storage value
Data is ho

Write a C-program to print average of first ten natural numbers.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i, sum = 0, avg;
    clrscr();
    for (i = 1, i <= 10; i++)
    {
        sum = sum + i;
    }
    avg = sum / 10;
    printf("%d", avg);
    getch();
}
```

Write a C-program to print table of any number

```
#include <stdio.h>
#include <conio.h>
void main
{
    int i, n, P;
    clrscr();
    printf("enter any no");
    scanf("%d", &n);
    for (i=1; i<=10; i++)
    {
        P = n * i;
        printf("%d", P);
    }
    getch();
}
```

C-program to print all even no. from 2 to n

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i, n;
    clrscr();
    printf("enter any no");
    scanf("%d", &n);
    for (i=2, i<=n, i++)
    {
        if (i%2 == 0)
        {
            printf("%d", i);
        }
    }
    getch();
}
```

* write a C-program to print factorial of a number

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i, n, f=1;
    clrscr();
    printf("enter any no");
    scanf("%d", &n);
    for (i=1, i<=n, i++)
```

```

{
    f = f * i
}
printf ("%d", f);
getch();
}

```

Ques * Write a C-program to print whether a number is prime or not it.

```

#include <stdio.h>
#include <conio.h>
void main()
{
    int i, n, f = 0;
    clrscr()
    printf ("enter any no");
    scanf ("%d", &n);
    for (i = 1, i <= n, i++)
    {
        if (n % i == 0)
        {
            f++;
        }
    }
    if (f == 2)
    {
        printf ("prime no");
    }
    else
    {
        printf ("Not prime No");
    }
    getch();
}

```

Syntax of while loop

```

intilization
while (condition)
{
    statement
    Increment
}
    
```

* First n natural number-

Ex Write a C-program to print n whole number by using while loop

* C-program to print n natural numbers by using while loop

* Sum of n natural no's by using while loop

* Sum of n whole number by using while loop

* Q.

Q1. Write a C-program to print n whole numbers by using while loop.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i, n;
    while (i <= 10)
    clrscr();
    printf("enter any number");
    scanf("%d", &n);
    i = 1;
    while (i <= n)
    {
        printf("%d", i);
        i++;
    }
    getch();
}
```

Q2. Write a C-program to print n natural numbers by using while loop.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i, n;
    for clrscr();
    printf("enter any number");
    scanf("%d", &n);
```

```

i = 1
while (i <= n)
{
    printf ("%d " i)
}
getch();
}

```

Sum of n natural numbers.

```

#include <stdio.h>
#include <conio.h>
void main()
{
    int i, n, sum = 0;
    clrscr();
    printf ("enter any number");
    scanf ("%d", &n);
    i = 1
    while (i <= n)
    {
        sum = sum + i;
        printf ("%d \n", sum);
        i++;
    }
    getch();
}

```

Q. Sum of n numbers by using while loop -

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int i, n, sum = 0;
```

```
    clrscr();
```

```
    printf("enter any number");
```

```
    scanf("%d", &n);
```

```
    i = 0
```

```
    while (i <= n)
```

```
    {
```

```
        sum = sum + i;
```

```
        printf("%d\n", sum);
```

```
        i++;
```

```
    }
```

```
    getch();
```

```
}
```

C-program to print sum of all even numbers.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int n, i, sum = 0;
    clrscr();
    printf ("enter any no");
    scanf ("%d", &n);
    for (i = 2, i <= n; i++)
    {
        if (i % 2 == 0)
        {
            printf ("%d", i);
            sum = sum + i;
        }
    }
    printf ("sum of all even number is %d", sum);
    getch();
}
```

Write a C-program to print ^{fibonacci} ~~key bonesis~~ series.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a = 0, b = 1, c, i, n;
    clrscr();
    printf ("enter any no");
    scanf ("%d", &n);
```

0 1 1 2 3 5 8 13

```

for (i=1; i<=n; i++)
{
    printf ("%d", a);
    c = a + b;
    a = b;
    b = c;
}
getch ();
}

```

* Write a c-program to print whether a number is perfect or not.

```

#include <stdio.h>
#include <conio.h>
{

```

```

    int n, i, sum = 0;
    clrscr ();

```

```

    printf ("enter any No");

```

```

    scanf ("%d", &n);

```

```

    for (i=1; i<n; i++)
    {

```

```

        if (n % i == 0)
        {

```

```

            sum = sum + i;
        }
    }

```

```

    if (sum == n)
    {

```

```

        printf ("perfect No");
    }

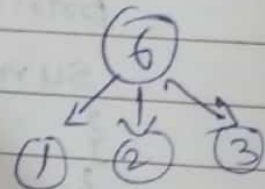
```

```

    else
    {
        printf ("Not perfect");
    }
    getch ();
}

```

2/6
3/3



Homework - 14/09/23

PAGE No.	
DATE	

Write a C-program to print sum of all odd numbers.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main ()
```

```
{
```

```
    int i, n, sum = 0;
```

```
    printf ("enter any number");
```

```
    scanf ("%d", &n);
```

```
    for (i = 1; i <= n; i++);
```

```
    {
```

```
        if (i % 2 != 0)
```

```
        {
```

```
            printf ("%d", i);
```

```
            sum = sum + i;
```

```
        }
```

```
    }
```

```
    printf ("sum of all evenodd number is %d", sum);
```

```
    getch();
```

```
}
```

Write a c program to print whether a number is armstrong or not.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int n, r, sum=0, armstrong;
    clrscr();
    printf ("enter any number\n");
    scanf ("%d", &n);
    armstrong = n;
    while (n>0)
    {
        r = n % 10; (remainder)
        sum = sum + r * r * r;
        n = n / 10 (quotient)
    }
    if (sum == armstrong)
    {
        printf ("armstrong No.");
    }
    else
    {
        printf ("Not armstrong No.");
    }
    getch ();
}
```

153	.	0	153
n	r	sum	armstrong

Write a C-program to print whether a number is strong or not.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int n, sum = 0, strong, r, fact, i;
```

```
clrscr();
```

```
printf("enter any no");
```

```
scanf("%d", &n);
```

```
strong = n;
```

```
while (n > 0)
```

```
{
```

```
    r = n % 10;
```

```
    fact = 1;
```

```
    for (i = 1; i <= r; i++)
```

```
    {
```

```
        fact = fact * i;
```

```
    }
```

```
    sum = sum + fact;
```

```
    n = n / 10;
```

```
}
```

```
if (strong == sum)
```

```
{
```

```
    printf("Strong number");
```

```
}
```

```
else
```

```
{ printf("Not strong number")
```

```
}
```

```
getch
```

```
}.
```

Nested loop - more than one loop

16/09/23

PAGE No.

DATE

Write a C program to print tables from 2 to 20.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int i, j, p, n;
    clrscr ();
    printf ("enter any No");
    scanf ("%d", &n);

    for (i = 2, i <= n; i++) outer, 2, 3, 4, 5, 6
    {
        for (j = 1; j <= 5; j++) inner loop
        {
            p = i * j;

            printf ("%d", p);
        }
        printf ("\n");
    }
    getch ();
}
```

Write a C program ^{for} star pattern

```
#include <stdio.h>
#include <conio.h>
void main ()
```

```

{
    int i, j;
    clrscr();
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < 3; j++)
        {
            printf("*");
        }
        printf("\n");
    }
    getch();
}

```

***	Star Pattern

Write a C-program to print sum of digits of a number.

```

#include <stdio.h>
#include <conio.h>

void main ()
{
    int n, r, sum = 0;
    clrscr();
    printf("enter any no");
    scanf("%d", &n);
    while (n > 0)
    {
        r = n % 10;
        sum = sum + r;
        n = n / 10;
    }
    printf("%d", sum);
    getch();
}

```

n	r	sum
---	---	-----

Write a program to print all prime numbers from a given range.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int min, max, i, j, count;
    clrscr();
    printf("enter values of min & max");
    scanf("%d %d", &min, &max);
    for (i = min; i <= max; i++)
    {
        count = 0;
        for (j = 1; j <= i; j++)
        {
            if (i % j == 0)
            {
                count++;
            }
        }
        if (count == 2)
        {
            printf("%d", i);
            getch();
        }
    }
}
```

Switch case statement :-

↓
default → keywords

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
{
```

```
int a, b, c, ch;
```

```
clrscr ();
```

```
printf ("enter value of a, b");
```

```
scanf ("%d %d", &a, &b);
```

```
printf ("enter your choice");
```

```
scanf ("%d", &ch);
```

case के लिए

```
switch (ch)
```

```
{
```

```
case 1:
```

```
c = a + b
```

```
printf ("%d", c);
```

```
break;
```

```
case 2:
```

```
c = a - b
```

```
printf ("%d", c);
```

```
break;
```

```
case 3:
```

```
c = a * b;
```

```
printf ("%d", c);
```

```
break;
```

```
default:
```

→ case के अंत में

```
printf ("wrong choice");
```

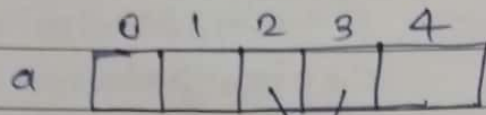
```
}
```

```
getch (); }
```

Array

Array:- collection of similar data type.

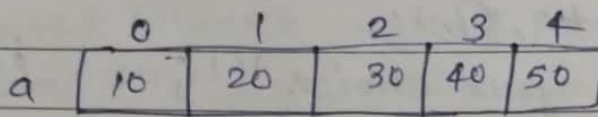
(i) `int a[5];`



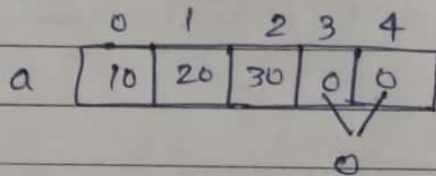
Garbage value.

int
float
char
Syntax —
data type array name [size]
int a [10]

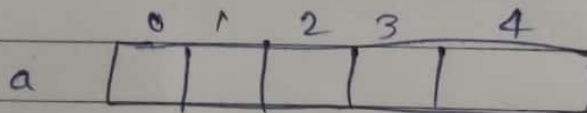
(ii) `int a[5] = {10, 20, 30, 40, 50};`



(3) `int a[5] = {10, 20, 30};`



(4) `int a[5] = {10, 20, 30, 40, 50, 60, 70};`



error.

Sample-1

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a[5] = {10, 20, 30, 40, 50};
    clrscr();
```

```

printf ("%d", a[0]);
printf ("%d", a[1]);
printf ("%d", a[2]);
printf ("%d", a[3]);
printf ("%d", a[4]);
getch();
}

```

PAGE No.				
DATE		2	3	4

a	10	20	30	40	50
---	----	----	----	----	----

10, 20, 30, 40, 50

sample-2

```

#include <stdio.h>
#include <conio.h>
void main
{
    int a[5] = {10, 20, 30, 40, 50}, i;
    clrscr

```

	0	1	2	3	4
a	10	20	30	40	50

i	☐
---	--------------------------

10
20
30
40
50

```

for (i=0; i<5; i++)
{
    printf ("%d\n", a[i]);
}
getch();
}

```

sample-3

Sample - 3

	0	1	2	3	4
a	10	20	30	40	50

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a[5], i;
    clrscr ();
    printf ("enter 5 integers");
    for (i=0; i<=4; i++)
    {
        scanf ("%d", &a[i]);
    }

    printf ("The entered value are as").--
    for (i=0; i<=4; i++)
    {
        printf ("%d", a[i]);
    }
    getch ();
}
```

0

i

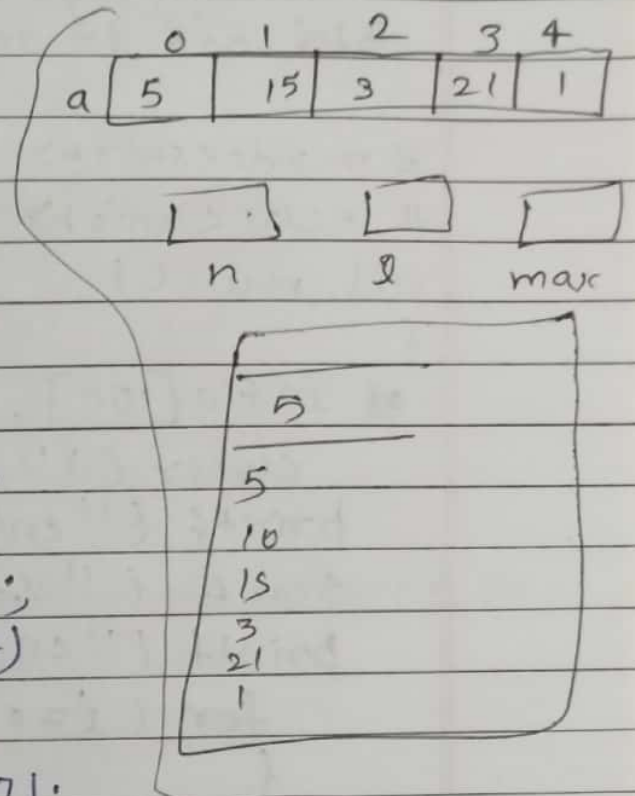
enter 5 integers
10
20
30
40
50
The entered values are
10 20 30 40 50

Largest No in Array - write a c program to find the largest No. in an Array :-

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a[100], n, i, max;
    clrscr ();
    printf ("enter any size");
    scanf ("%d", &n);
    printf ("enter data");
    for (i=0; i<n; i++)
    {
        scanf ("%d", &a[i]);
    }

    max = a[0];
    for (i=1; i<n; i++)
    {
        if (a[i] > max)
        {
            max = a[i];
        }
    }

    printf ("The largest element is %d", max);
    getch ();
}
```



Write a C-program to find the total even and total odd in an array:-

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a[100], n, i, count1 = 0, count2 = 0;
    clrscr ();
    printf ("enter size");
    scanf ("%d", &n);
    printf ("enter data");
    for (i = 0; i < n; i++)
    {
        scanf ("%d", &a[i]);
    }

    for (i = 0; i < n; i++)
    {
        if (a[i] % 2 == 0)
        {
            count1++;
        }
        else
        {
            count2++;
        }
    }

    printf ("The total evens are as %d", count1);
    printf ("The total odds are as %d", count2);
    getch ();
}
```

Sorting :-

- Bubble sort
- Selection sort
- Insertion sort
- Merge sort
- Quick sort -

Sorting is a process to arrange the element in ascending order.

Bubble sort

	0	1	2	3	4
a	15	16	6	8	5

↑ ↑

15 16 6 8 5

↑ ↑

15 6 16 8 5

↑ ↑

15 6 8 16 5

↑ ↑

15 6 8 5 16

pass 0

pass - 1

15 6 8 5 16

↑ ↑

6 15 8 5 16

↑ ↑

6 8 15 5 16

↑ ↑

6 8 5 15 16

↑ ↑

6 8 5 15 16

pass-2

6 8 5 15 16
 6 8 5 15 16
 6 5 8 15 16

pass-3

6 5 8 15 16
 5 6 8 15 16

Write a C program to sort an array by using bubble sort.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[100], n, i, j, temp;
    clrscr();
    printf("enter size");
    scanf("%d", &n);
    printf("enter data");
    for (i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }
}
```

```
for (j=1; j<n; j++)
```

if ($a[j] < a[j-1]$)

$$\{$$

```
temp = a[j];
```

$$a[j] = a[j-1];$$
$$a[j-1] = \text{temp};$$

3

3

3

```
printf ("The sorted array is=");
```

```
for (i=0; i<n; i++)
```

{

```
printf ("%d", a[i]);
```

3

```
getch();
```

3

selection sort -

a

0	1	2	3	4	5
7	4	10	8	3	1

1 4 10 8 3 7

↑ ↑

1 3 10 8 4 7
↑ ↑

1 3 4 8 10 7

1 3 4 7 10 8
 ↑

1	3	4	7	8	10
---	---	---	---	---	----

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a[100], n, i, j, temp;
    printf ("enter size");
    scanf ("%d", &n);
    printf ("enter data")
    for (i=0; i<n; i++)
    {
        scanf ("%d", &a[i]);
    }

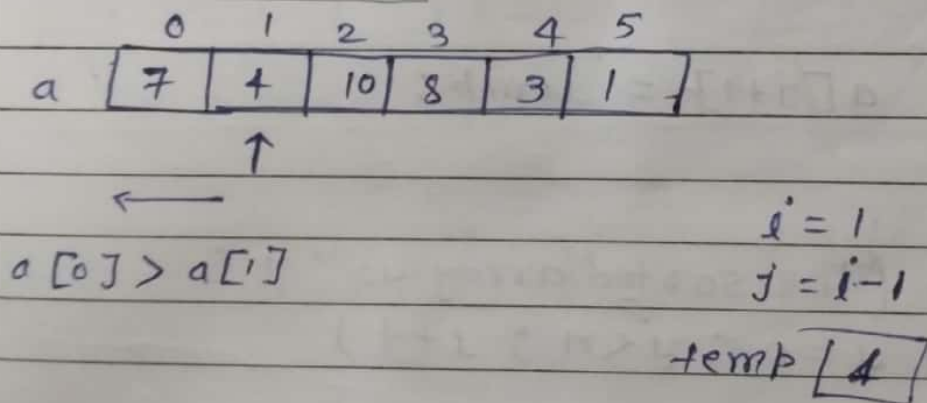
    for (i=0; i<n; i++)
    {
        min = i;

        for (j=i++ ; j<n; j++)
        {
            if (a[j] < a[min])
            {
                min = j;
            }
        }

        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
}
```

```
printf("the sorted array is = ");
for (i=0; i<n; i++)
{
    printf("%d", a[i]);
}
getch();
}
```

insertion sort:-



```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[100], i, j, temp, n;
    clrscr();
    printf("enter size");
    scanf("%d", &n);
```

PAGE No.
 DATE
 printf (" enter data")
 for (~~i~~ i=0; i<n; i++)
 {
 scanf ("%d", &a[i]);
 }

for (i=1; i<n; i++)
 {

 temp = a[i];

 j = i-1;

 while (j>=0 && a[j] > temp)
 {

 a[j+1] = a[j];

 j = j-1;

 }

 a[j+1] = temp;

 }

printf ("The sorted array is");

for (i=0; i<n; i++)
 {

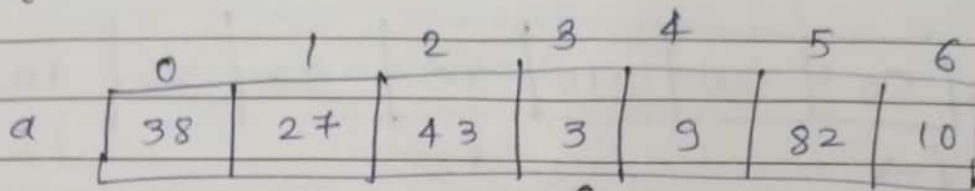
 printf ("%d", a[i]);

}

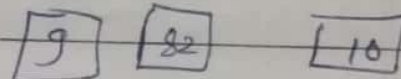
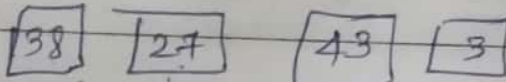
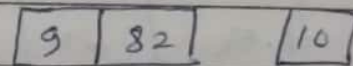
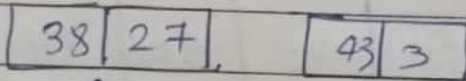
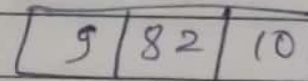
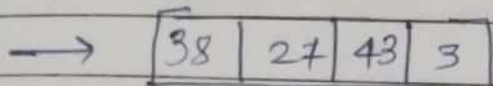
 getch();

}

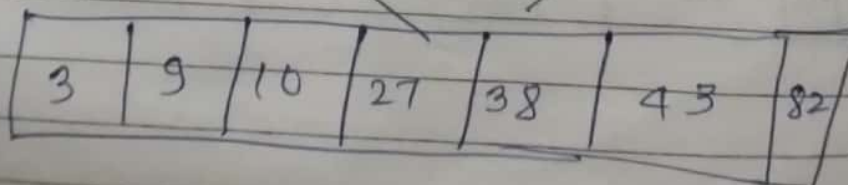
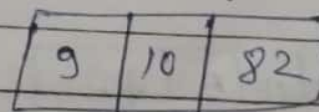
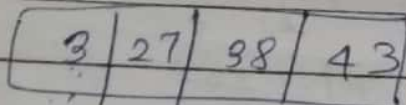
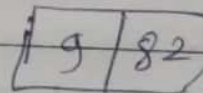
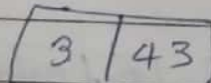
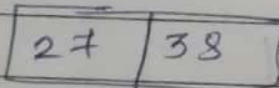
Merge Sort:-



Divide

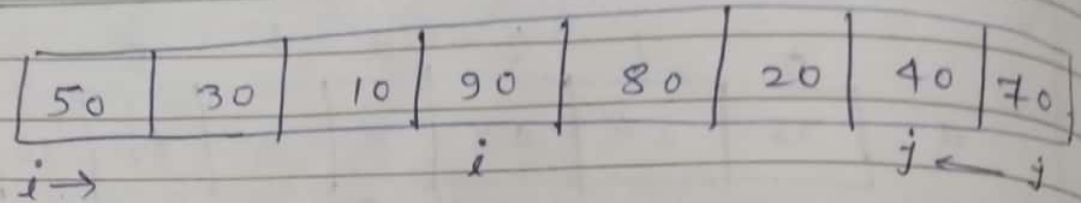


Sorting -



imp

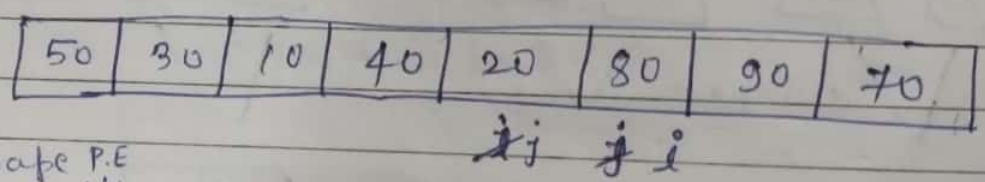
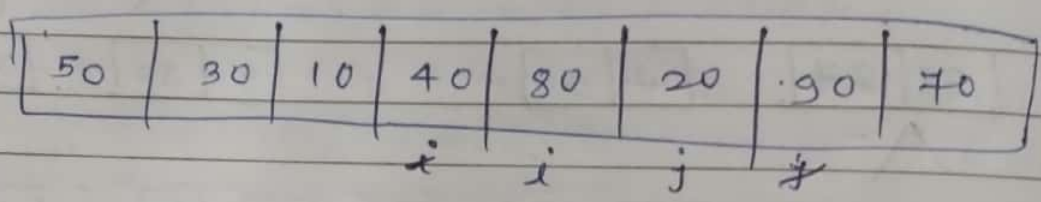
Quick sort



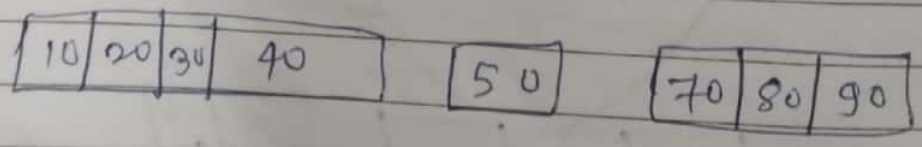
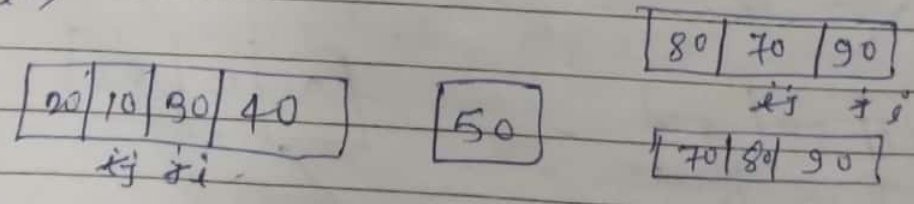
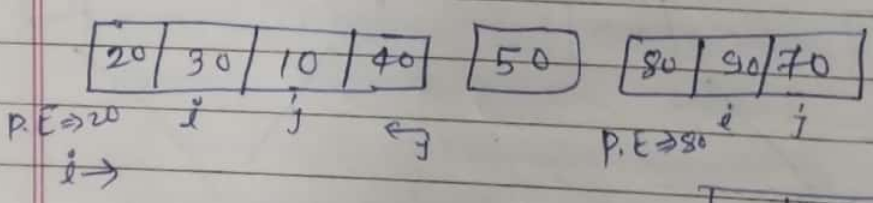
pivot element $\rightarrow$ 1st element

$i^{th} \rightarrow l \text{ to } R \rightarrow \text{Greater than P.E}$
 $j^{th} \rightarrow R \text{ to } l \rightarrow \text{Smaller than P.E}$

P.E $\rightarrow$ 50



swap P.E
(j)



Assignment-1

PAGE No.	
DATE	

- ① Define C Language. Explain history of C language.
- ② Define C tokens. Explain in detail.
- ③ Describe operators which are used in C-language.
- ④ Define (i) keywords (ii) identifiers (iii) Data types
(iv) input output statement (v) variables (vi) constant
- ⑤ Difference b/w compiler and interpreter.
- ⑥ Define ^{arr} Array. Explain one-D and 2-D Array.
- ⑦ Define allocation. Explain static and dynamic allocation.

Assignment - 1

Q1. Define C-Language. Explain history of C language.

C-Language :- The C programming language is a procedural and general-purpose language that provides low-level access to system memory.

History of C-language-

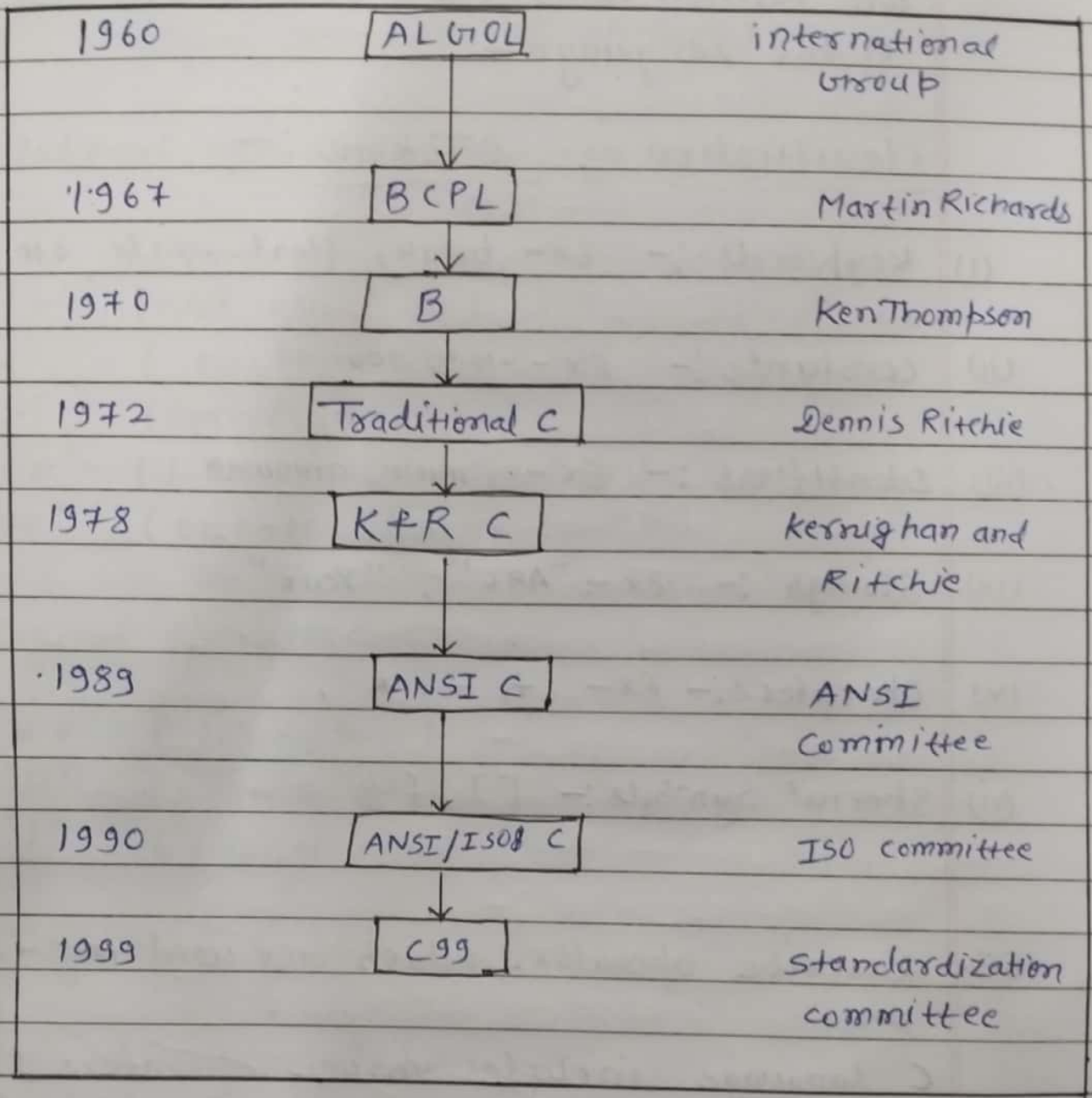
C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & telegraph), located in the U.S.A.

Dennis Ritchie is known as the founder of the C language.

It was developed to overcome the problems of previous languages such as B, BCPL, etc.

Initially, C language was developed to be used in UNIX operating system. It inherits many features of previous language such as B and BCPL.

Let's see the programming language that were developed before C language-



Q2. Define C tokens. Explain in detail.

C Tokens :-

In a passage of text, individual words and punctuation marks are called tokens.

Similarly, in a C program the smallest individual units are known as C tokens.

Chas. Six types of tokens • C program

are written using these tokens and the syntax of the language -

Classification of C Tokens :-

- (i) Keywords :- Ex- break, float, while etc.
- (ii) Constants :- Ex- -15.5, 100
- (iii) Identifiers :- Ex- main, amount
- (iv) Strings :- Ex- "ABC", "Year"
- (v) Operators :- Ex- +, -, *,
- (vi) Special Symbols :- [], { }

Q3. Describe operators which are used in C-language.

C language includes various operators that allow you to perform operations on data and variables.

Here's an overview of some of the commonly used operators in C:

1. Arithmetic operators :-

- '+' (Addition)
- '-' (Subtraction)
- '*' (multiplication)

'/' (Division)

'%' (Modulus, remainder of division)

2. Relational operators :-

'==' (Equal to)

'!=' (Not equal to)

'<' (less than)

'>' (greater than)

'<=' (Less than or equal to)

'>=' (greater than or equal to)

3. Logical operators :-

'&&' (Logical AND)

'||' (Logical OR)

'!' (Logical NOT)

4. Assignment operators :-

'=' (Assignment)

'+=' (Addition Assignment)

'-=' (Subtraction Assignment)

'*=' (Multiplication Assignment)

'/=' (Division Assignment)

'%=' (Modulus Assignment)

5. Increment/Decrement operators :-

'++' (Increment by 1)

'--' (Decrement by 1)

6. Bitwise operators :-

- '&' (Bitwise AND)
- '|' (Bitwise OR)
- '^' (Bitwise XOR)
- '~' (Bitwise NOT)
- '<<' (Left shift)
- '>>' (Right shift)

7. Conditional (Ternary) operators :-

- '?:' (conditional operator, used for simple conditional expressions)

8. Comma operator :-

- ',' (comma operator used to separate expressions)

9. Size of operator :-

- 'sizeof' (used to determine the size in bytes of a data types or variable)

10. Pointer operators :

- '*' (Dereference operator, used to access the value pointed to by a pointer)
- '&' (Address-of operator, used to obtain the address of a variable)

These operators are fundamental for

performing various operations in C Programming. Depending on the context and the data types involved, you can use these operators to manipulate and evaluate expressions in your C programs.

Q4. Define (i) Keywords (ii) Identifiers (iii) Data types (iv) input output statement (v) variables (vi) constant

(i) Keywords:- All keywords have fixed meanings and these meanings cannot be changed. Keywords serve as basic building blocks for program statement.
Ex:- break, case, if, else, switch etc.

(ii) Identifier:- Identifier in C language represent the names of various entities such as arrays, functions, variables, user-defined data types, labels etc.

Rules for identifier:-

- (1) First character must be an alphabet (or underscore).
- (2) Must consist of only letters, digits or underscore.
- (3) Only first ~~25~~ 31 characters are significant.
- (4) Cannot use a ~~key~~ keyword.
- (5) Must not contain white space.

(iii) Data types :- C language is rich in its data types. storage representation and machine instructions for

(iii) Data types :- It refers to the type of value a variable has & what type of mathematical, relational or logical operations can be applied without causing an error.

(iv) Input/output Statement :- An input/output statement or IO Statement is a portion of a program that instructs a computer how to read & process data.

(v) Variables :- It is the name of a memory location that we use for storing data. We can change the value of a variable in C or any other language & we can also reuse it multiple times.

(vi) Constants :- The constants in C are the read-only variable whose values can't be modified once they are declared in the C-program. In C language the 'const' keyword is used to define the constants.

Q5. Difference b/w compiler & interpreter?

PAGE No.
 DATE

Compiler:- It is translator which takes input as high level language & produces an output of low-level language i.e machine or assembly language. The work of a compiler is to transform the codes written in programming language into machine code (0 or 1) so that computers can understand it.

Interpreter:- An interpreter is a program that translates a programming languages into a comprehensible language. The interpreter converts high-level language to an intermediate language. It contains pre-compiled code, source code etc. It translates only one statement of program at a time.

Q6. Define Array. Explain 1-D & 2-D array?

Ans. Array in C is one of the most used data structures in C-programming. It is a simple & fast way of storing multiple values under a single name. Array is a fixed size collection of similar data items stored in continuous memory locations.

1-D Array:-

A one dimensional array is a structured collection of components, often called array elements that can be access

individually by specifying the position of a computer component with a single index value. An array variable must be declared before being used in a program.

2-D Array :- A two dimensional array or 2-D array in C is the simplest form of the multidimensional array. We can visualize a two dimensional array as an array of one dimensional array arranged one over another forming a table with 'x' rows & 'y' columns where the row number ranges from 0 to (x-1) & column no ranges from 0 to (y-1).

Q7. Define memory allocation. Explain static & dynamic memory allocation.

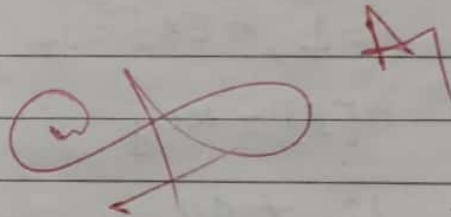
Ans. Memory Allocation :- Memory allocation is the process of reserving virtual or physical computer space for a specific-purpose (e.g. for computer programs & services to run). Memory allocation is part of the management of computer memory resources, known as memory management.

* Static Memory Allocation :- It is a process in which memory for variables & data structures is reserved at compile time, before the program

runs. This means that the size & location of memory blocks are fixed & can't be change at runtime.

* Dynamic memory allocation:- It is a procedure in which size of a data structure (st. like array) is changed during runtime. There are 4 library functions provided by C defined under `<stdio.h>` header file to facilitate dynamic memory allocation in C-programming. They are -

- (i) `malloc()`
- (ii) `calloc()`
- (iii) `free()`
- (iv) `realloc()`



Searching → To find element

→ Linear search

→ Binary search

Linear search →
(unsorted array)

	0	1	2	3	4
a.	13	12	4	90	3

$i = 0, a[i] = 13$

~~13~~ 90
key

$a[i] = \text{key}$

$13 \neq \text{key } 90$
 $i++$

$i = 1, a[i] = 12$

$a[i] = \text{key}$

$12 \neq 90$

$i++$

$i = 2, a[i] = 4$

$a[i] = \text{key}$

$4 \neq 90$

$i++$

se.

$i = 3, a[i] = 90$

$a[i] = \text{key}$

$90 = 90$

Search Success

Binary Search :- (only for use sorted array)

	0	1	2	3	4	5	6	7	8	9
a	10	20	30	40	50	60	70	80	90	100

60
key

low $\rightarrow$ starting index = 0
high $\rightarrow$ ending index = 9

$$\text{mid} = \frac{\text{low} + \text{high}}{2} = \frac{0 + 9}{2} = \frac{9}{2} \Rightarrow 4$$

$$a[4] = \text{key}$$

$$50 \neq 60$$

$$\text{key} > a[\text{mid}]$$

$$\text{low} \rightarrow \text{mid} + 1 \Rightarrow 4 + 1 = 5$$

$$\text{high} \rightarrow \text{high} \Rightarrow 9$$

$$\text{mid} = \frac{\text{low} + \text{high}}{2} = \frac{5 + 9}{2} = \frac{14}{2} = 7$$

$$a[7] = \text{key}$$

$$80 \neq 60$$

$$\text{key} < a[\text{mid}]$$

$$\text{low} = 5$$

$$\text{high} = \text{mid} - 1 = 7 - 1 = 6$$

$$\text{mid} = \frac{\text{low} + \text{high}}{2}$$

$$= \frac{5 + 6}{2} = \frac{11}{2} \Rightarrow 5$$

$$a[5] = \text{key}$$

$$60 = 60$$

Search Success

2D-Array (Two dimensional array).

rows columns.

syntax

Datatype	Array name [row size] [column size]
int	A [100 ₃] [100 ₄] =
float	= 1, 2, 3, 4, 5, 6, 7, 8, 9
char	10, 11, 12

		0	1	2	3
0		1	2	3	4
1		5	6	7	8
2		9	10	11	12

max value

Data यदि कम होगा तो, उसके स्थान पर 0 आ जाएगा।
 Data यदि अधिक होगा तो error आ जाएगा।

Sample program of 2D-Array

include <stdio.h>

include <conio.h>

void main ()

{

int a[3][3], i, j;

clrscr();

printf ("enter data in an 2D Array");

for (i=0; i<3; i++)

{

for (j=0; j<3; j++)

{

scanf ("%d", &a[i][j]);

}

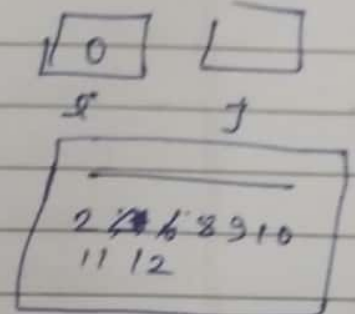
}

for
enter
data

Displaying result —

```
for (i=0; i<3; i++)  
{  
    for (j=0; j<3; j++)  
    {  
        printf ("%d", a[i][j]);  
    }  
    getch();  
}
```

	PAGE No.		
	DATE	0	1
		2	
0	2	4	6
1	8	9	10
2	11	12	13



Sample program —

```
#include <stdio.h>  
#include <conio.h>  
void main()  
{  
    int a[3][2], b[3][2], i, j;  
    clrscr();  
    printf ("enter data for a=");  
    for (i=0; i<3; i++)  
    {  
        for (j=0; j<2; j++)  
        {  
            scanf ("%d", &a[i][j]);  
        }  
    }  
    printf ("enter data for b=");  
    for (
```

impAddition of two matrix

```

#include <stdio.h>
#include <conio.h>
void main()
{
    int A[100][100], B[100][100], C[100][100], r, c, i, j;
    clrscr();
    printf("Enter row size");
    scanf("%d", &r);
    printf("Enter column size");
    scanf("%d", &c);
    printf("Enter data in matrix A");
    for (i=0; i<r; i++)
    {
        for (j=0; j<c; j++)
        {
            scanf("%d", &A[i][j]);
        }
    }
    printf("Enter data in matrix B");
    for (i=0; i<r; i++)
    {
        for (j=0; j<c; j++)
        {
            scanf("%d", &B[i][j]);
        }
    }
}

```

Logic of add-

```

for (i=0; i<r; i++)
{
    for (j=0; j<c; j++)

```

```

{
    c[i][j] = A[i][j] + B[i][j]
}
}

```

For displaying -

```

printf("The addition of two matrix is");
for(i=0; i<r; i++)
{
    for(j=0; j<c; j++)
    {
        printf("y.d", c[i][j]);
    }
}

getch();
}

```

Multiplication of two Matrix :- by using c-language

```

#include <stdio.h>
#include <conio.h>
void main()
{
    int a[100][100], b[100][100], c[100][100], r, c, i, j, k, sum=0;
    clrscr();
    printf("enter row size");
    scanf("%d", &r);
    printf("enter column size");
    scanf("%d", &c);
}

```

```
printf (" enter data in Matrix A")
```

```
{
for (i=0; i<r; i++)
```

```
{
for (j=0; j<c; j++)
```

```
{
```

```
scanf ("%d", &a[i][j]);
```

```
}
```

```
}
```

```
printf (" enter data in matrix B");
```

```
{
for (i=0; i<r; i++)
```

```
{
for (j=0; j<c; j++)
```

```
{
```

```
scanf ("%d", &b[i][j]);
```

```
}
```

```
}
```

logic of Multiplication -

```
for (i=0; i<r; i++)
```

```
{
```

```
for (j=0; j<c; j++)
```

```
{
```

```
for (k=0; k<r; k++)
```

```
{
```

```
sum = sum + a[i][k] * b[k][j];
```

```
}
```

```
c[i][j] = sum
```

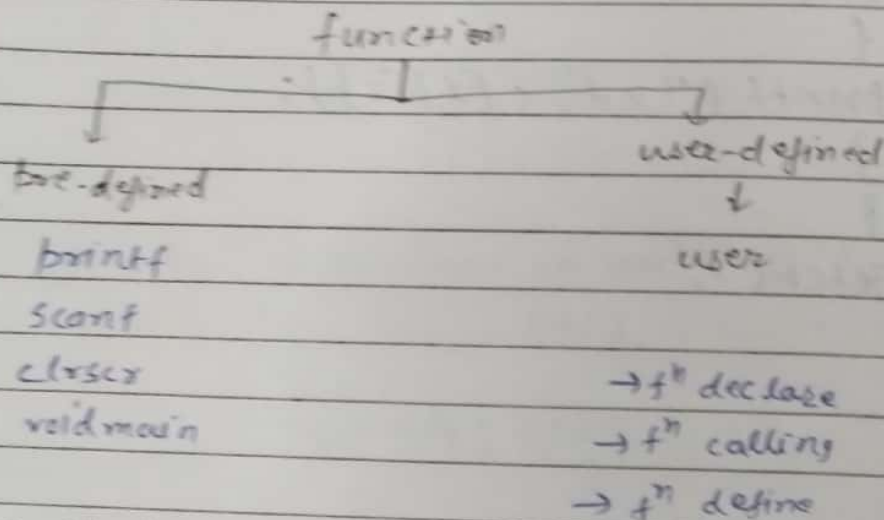
```
sum = 0;
```

```
}
```

```
}
```

```
printf("The multiplication of two matrix is");
for (i=0; i<8; i++)
{
    for (j=0; j<8; j++)
    {
        printf("%d", c[i][j]);
    }
}
getch();
}
```

function :- Set of blocks



Sample-1

Sub/Addition of two numbers by using fⁿ.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void add();
```

```
void sub();
```

```
void main()
```

```
{
```

```
clrscr();
```

```
add();
```

```
sub();
```

```
printf("addition & sub complete");
```

```
getch();
```

```
}
```

```
void add()
```

```
{
```

```
int a, b;
```

```
printf("enter values of a & b");
```

(fn declare)

{ fⁿ calling }

{ fⁿ define }

```
scanf ("%d %d", &a, &b);
c = a + b;
printf ("%d", c);
}
```

```
void sub ()          {fn define}
{
    int d, e, f;
    printf ("enter the values of d & e");
    scanf ("%d %d", &d, &e);
    f = d - e;
    printf ("%d", f);
}
```

Sample-2 area & perimeter of circle & rectangle by using fn

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a, s;
    clrscr ();
    printf ("enter side");
    scanf ("%d", &s);
    a = s * s;
    printf ("The area is %d", a);
    circle ();
    rectangle ();
    getch ();
}

void circle ()
{
}
```

```

int r, a, p;
printf ("enter radius");
scanf ("%d", &r);
a = 3.14 * r * r;
printf ("%d", a);
p = 2 * 3.14 * r;
printf ("%d", p);

```

```

void rectangle()
{
    int l, b, a, p;
    printf ("enter l & b");
    scanf ("%d %d", &l, &b);
    a = l * b;
    printf ("%d", a);
    p = 2 * (l + b);
    printf ("%d", p);
}

```

Sample-3

```

#include <stdio.h>
#include <conio.h>
void circle();
void rectangle();
void main()
{
    clrscr();
    circle();
    getch();
}

```

```
void circle()
```

```
{
```

```
    ;
```

```
    ;
```

```
    ;
```

```
    ;
```

```
    ;
```

```
    ;
```

```
}
```

```
void rectangle()
```

```
{
```

```
    ;
```

```
}
```

8. write a c-program to print addition of two numbers by using function through passing parameters.

```
#include <stdio.h>
#include <conio.h>
void add (int, int);
void main()
{
    int x, y;
    clrscr();
    printf("enter value of x+y = ");
    scanf ("%d %d", &x, &y);
    add(x, y);
    getch();
}

void add (int a, int b)
{
    int c;
    c = a + b;
    printf ("%d", c);
}
```

write a c-program to print swape of two numbers by using function through passing parameters.

```
#include <stdio.h>
#include <conio.h>
void swap (int, int);
void main()
{
    int x, y;
    clrscr();
    printf("enter value of x+y = ");
```

```
scanf ("%d %d", &x, &y);
swap (x, y);
getch ();
}
```

```
void swap (int a, int b)
{
    int c;
    c = a;
    a = b;
    b = c;
}
```

```
printf ("The value of a+b after swaping is %d %d",
a, b);
}
```

write a C-program to return the largest value among two numbers by using a function.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int greatest (int, int)
```

```
void main()
```

```
{ int x, y, z;
```

```
printf ("enter value of x+y =");
```

```
z = greatest (x, y);
```

```
printf ("%d", z);
```

```
getch ();
```

```
}
```

```
int greatest (int a, int b)
```

```
{
```

```
if (a > b)
```

```
return (a);
```

```
}
```

```
else {
```

```
return (b)
```

```
}
```

write a c-program to return the average of three numbers by using a function.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int avg (int, int, int);
```

```
void main()
```

```
{ int x, y, z, w;
```

```
clrscr();
```

```
printf ("enter data x, y, z = ");
```

```
scanf ("%d %d %d", &x, &y, &z);
```

```
w = Avg (x, y, z)
```

```
printf ("The average of three numbers is %d", w);
```

```
getch();
```

```
}
```

```
int avg (inta, intb, intc)
```

```
{
```

```
int t;
```

```
t = (a+b+c)/3 ;
```

```
return (t);
```

```
}
```

v v 2

PAGE No.	
DATE	

Write a C program to print sum of all natural numbers by using recursion.

```
int sum(int):
void main()
{
    int x, w;
    clrscr();
    printf("enter x");
    scanf("%d", &x);
    w = sum(x);
    printf("%d", w);
    getch();
}

int sum(int a)
{
    if (a == 1)
    {
        return (1);
    }
    else
    {
        return(a + sum(a-1));
    }
}
```

Fibonacci series by recursion.

```
#include <stdio.h>
#include <conio.h>
int fibo(int)
void main()
{
    int i, n, w;
    clrscr();
    printf("enter n");
    scanf("%d", &n);
    for (i=0; i<n; i++)
    {
        w = fibo(i)
        printf("%d", w)
    }
    getch();
}

int fibo(int a)
{
    if (a == 0)
    {
        return (0);
    }
    else
    {
        if (a == 1);
        {
            return (1);
        }
        else
        {
            return (fibo(a-1) + fibo(a-2));
        }
    }
}
```

* Write a C program to print sum of whole number by using recursion.

```
#include <stdio.h>
#include <conio.h>
int x, w;
class ();
printf ("enter x");
scanf ("%d", &x);
w = sum (x);
printf ("%d", w);
getch ();
}
```

```
int sum (int a)
{
    if (a == 0)
    {
        return (0);
    }
    else
    {
        return (a + sum (a-1));
    }
}
```

String:- group of characters

\0 ⇒ Null character

'w' = w

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
{
    char A[10] = "wajahad"
    clrscr();
    printf("Y.S", A);
    getch();
}
```

sample - 2

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
{
    char A[10]
    clrscr();
    printf("Enter any string");
    scanf("Y.S", A);
    printf("Y.S", A);
    getch();
}
```

Sample - 3

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
{
```

```

char A[20]
clrscr()
printf("enter string")
    gets(A);           ← replace scanf
printf("Y.S", A);
getch();
}
    
```

```

scanf = gets ( )
printf = puts(A)
    
```

imp

Standard library functions of String

Pr-Wajahat

- i. strlen → No. of digits → 7
- (ii) strcpy → copy of a string →
- (iii) strcmp → comparison → return 0 → equal
- (iv) strrev → reverse of a string → tahajan ex: watah

prac

factorial by using recursion:-

```

#include <stdio.h>
#include <conio.h>
int fact(int)
void main()
{
    int n, f;
    clrscr();
    printf("enter any no");
    scanf("%d", &n);
    f = fact(n);
    printf("%d", f);
    getch();
}
    
```

```

int fact (int X)
{
    if (x == 1)
    {
        return 1;
    }
    else
    {
        return (x * fact (x-1));
    }
}

```

Ackerman f^n by using recursion:-

$$A(m, n) = ?$$

(i) If $m = 0$, $n+1$

(ii) If $m > 0$ & $n = 0$;

(iii) If $(m > 0$ & $n > 0$; $A(m-1, A(m, n-1))$;

Ex -

$$A(0, 1) = 1 + 1 = 2$$

$$A(1, 0) \Rightarrow A(0, 1) \Rightarrow 2$$

$$A(1, 2) \Rightarrow A(0, A(1, 1))$$

$$\downarrow$$

$$A(0, A(1, 0))$$

$$\downarrow$$

$$A(0, 1)$$

$$A(1, 3)$$

$$\underline{\quad 4 \quad}$$

$$A(0, 2)$$

$$\underline{\quad 3 \quad}$$

```
#include <stdio.h>
#include <conio.h>
int Ack (int, int)
void main()
{
    int m, n, A;
    clrscr();
    printf ("Enter m+n ");
    scanf ("%d %d", &m, &n);
    A = ack (m, n);
    printf ("%d", A);
    getch();
}
int Ack (inta, intb)
{
    if (a == 0)
    {
        return b+1;
    }
    else
    {
        if (a > 0 && b == 0)
        {
            return ack (a-1, 1);
        }
        else
        {
            return ack (a-1, ack (a, b-1));
        }
    }
}
```

String standard library fn:- (sample program)

(i) strlen

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
{
    char A[20];
    int x;
    clrscr();
    printf("enter any string");
    scanf("%s", A);
    x = strlen(A);
    printf("The value of x is %d", x);
    getch();
}
```

(ii) strcpy-

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
char A[20], B[20];
clrscr();
printf("enter any string");
scanf("%s", A);
strcpy(B, A);
printf("%s", B);
getch();
}
```

(iii) strcmp

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
{
    char A[20];
    clrscr();
    printf("enter any string");
    scanf("%s", A);
    strcmp(A)
    printf("%s", A);
    getch();
}
```

(iv) strcmp

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
{
    char A[20], B[20]; int x;
    clrscr();
    printf("enter string");
    scanf("%s", A);
    printf("enter another string");
    scanf("%s", B);
    x = strcmp(A, B);
    if (x == 0)
    {
        printf("equal string");
    }
}
```

```

else
{
printf ("Not equal" );
}
getch ();
}

```

V.V.I
imb

* Define string. explain standard library fⁿ of a string. write a c-program to print whether a string is a palindrome or not.

```

#
#
#
void main()
{
char A[50], B[50]
int x;
clrscr ();
printf ("enter any string")
scanf ("%s", A);
strcpy (B, A);
strrev (B);
x = strcmp (A, B)
if (x == 0 )
{
printf ("palindrome string");
}
else
{
printf ("Not palindrome string");
}
getch ();
}

```

Q. Define String • Explain standard library function of string. Write a C-program to print whether a string is a palindrome or not.

Ans. String:- A string in C programming is a sequence of characters terminate with null character '\0'.

The C string is stored as an array of characters. The difference b/w a character array and C string is the string is terminated with a unique character '\0'.

String library function:-

The pre defined functions which are designed to handle strings are available in the library string.h. They are:-

(i) The strlen () function:-

It returns the number of characters in a string.

Syntax:- int strlen (String name)

Ex -

```
#include <string.h>
```

```
void main()
```

```
{
```

```
    char A[20];
```

```
    int x;
```

```
    clrscr();
```

```
    printf("enter any string");
```

```
    scanf("%s", A);
```

```
x = strlen(A)
```

```
printf("The value of x is %d", x);
getch();
}
```

~~output~~

(ii) The strcpy() function:-

- It is for copying source string into destination string
- The length of the destination string $\geq$ source string

Syntax (Destination string, source string)

ex:-

```
#include <string.h>
```

```
void main()
```

```
{
```

```
char a[50], b[50]
```

```
printf("enter a source string");
```

```
scanf("%s", a);
```

```
x printf("enter a destination string"); x
```

```
x scanf("%s", b); x
```

```
strcpy(b, a);
```

```
printf("copied string = %s", b);
```

```
getch();
```

```
}
```

out put:-

Enter a source string - Hello

copied string = Hello

(iii) The strrev() function

- The function is used for reversing a string.
- The reversed string will be stored in the same string.

Syntax^{strrev}(string)

Ex:-

```
#include <stdio.h>
main()
{
    char a[50];
    clrscr();
    printf("enter a string");
    gets(a);
    strrev(a);
    printf("reversed string = %s", a);
    getch();
}
```

output:-

enter a string Hello
reversed string = olleH

(iv) The strcmp() function (String comparison)

- This function compares 2 strings.
- It returns the ASCII difference of the first two non-matching characters in both the strings.

syntax:-

```
int strcmp (String 1, String 2);
// If the difference is equal to zero, then String 1 = String 2
// If the difference is negar positive the String 1 > String 2
// If the difference is negative, then String 1 < String 2.
```

ex:-

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
void main()
char A[20], B[20];
int x;
clrscr();
printf("enter string");
scanf("%s", A);
printf("enter another String");
scanf("%s", B);
x = strcmp(A, B);
if (x == 0)
{
printf("equal string");
}
else
{
printf(not("not equal string"));
}
getch();
}
```

* String a palindrome or not:-

```
#include <stdio.h>
#include <conio.h>
#include <string string.h>
void main()
{
    char A[50], B[50];
    int x;
    printf("enter any string");
    scanf("%s", A);
    strcpy(B, A);
    strrev(B);
    x = strcmp(A, B);
    if (x == 0)
    {
        printf("palindrome string");
    }
    else
    {
        printf("Not palindrome string");
    }
    getch();
}
```

pointers

↳ special type variable which stores address of any variable.

int a = 5 5
 1000 a
 int * B
 B = &a 1000
 B

float b = 10 10
 2000 b
 float * C;
 C = &b 2000
 C

char d = 'A' 'A'
 1500 d
 char * f;
 f = &d; 1500
 f

$\left\{ \begin{array}{l} \text{int * P;} \\ \text{int * P;} \\ \text{or} \\ \text{int * P;} \end{array} \right\}$

value
* P (1000)

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int a;
```

```
int * P;
```

```
clrscr();
```

```
printf("enter value of a");
```

```
scanf(" %d", &a);
```

```
P = &a;
```

```
printf("%d", a);
```

```
printf("%d", P);
```

```
printf("%d", &a);
```

```
printf("%d", &P);
```

```
printf("%d", *P);
```

```
printf("%d", *(*&P));
```

```
getch();
```

```
}
```

V.V.I

call by value →

#include <st

V.V.I

Define function. Difference b/w call by value and call by reference/address. Write a C-Program to print swape of two numbers using call the value & call the reference.

Function:-

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function. Functions are used to perform certain actions, and they are important for reusing code. Define the code once, and use it many times.

- The parameters passed to the function are called actual parameters whereas the parameters received by the function are called formal parameters.

call by value :-

- In the call by value method, there is no modification in the original value.
- There are two copies of parameters stored in different memory locations.
- One is the original copy and the other is the function copy.

- Any changes made inside functions are not reflected in the actual parameters of the caller.

* c-program to print swape of two numbers using call by value:-

```
#include <stdio.h>
#include <conio.h>
int swap(int, int);
void main()
{
    int a, b;
    clrscr();
    printf("enter the value of a & b");
    scanf("%d %d", &a, &b);
    swap(a, b);
    printf("The value of a & b %d %d", a, b);
    getch();
}

int swap(int p, int q)
{
    int temp;
    temp = p;
    p = q;
    q = temp;
    printf("The value of p & q %d %d", p, q);
}
```

output:-

inside function:-

p = 20, q = 10

In the caller:

a = 10, b = 20

• Call by Reference :-

4

- In the call by reference method, there is a modification in the original value.
- In call by reference method of parameter passing, the address of the actual parameters is passed to the function as the formal parameters.
- Both the actual and formal parameters refer to the same locations.
- Any changes made inside the function are actually reflected in the actual parameters of the caller.

* C-program to print swap of two numbers using call by reference/address.

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a, b;
    clrscr();
    printf ("enter value of a, b");
    scanf ("%d %d", &a, &b);
    swap (&a, &b);
    printf ("The value of a & b is %d %d", a, b);
    getch();
}
```

27/10/23

Q.

```
int swap(int* P, int* Q)
{
```

```
    int temp;
```

```
    temp = *P;
```

```
    *P = *Q;
```

```
    *Q = temp;
```

```
    printf("The value of P & Q are as %d %d", *P, *Q);
```

output:-

inside the function

p = 20 y = 10

inside the caller:

a = 20, b = 10

Thus actual values of a and b get changed after exchanging values of x and y.

27/10/23

Q. Define pointer, what are the uses of pointer. explain the types of pointer

Pointer:- The pointers in C language refer to the variables that hold the addresses of different variables of similar type data types.

uses of pointer:-

- To pass arguments by reference.
- For accessing array element.

- #

void pointer:- It is a special type of pointer that can be pointed at objects of any data type. A void pointer is declared like a normal pointer, using the void keywords as the pointer's type.

Ex:-

int a = 5	
float b = 5.5	
char c = 'A'	

int *ip → ip = &a
float *fp → fp = &b
char *cp → cp = &c

c = &b (compilation error)

The above problem can be solved by general purpose pointer called void pointer.

void pointer can be declared as follows:-

void * (int *) vp;
void * (float *) vp;

• Dangling pointer:-

If any pointer is pointing the memory address of any variable but after some variable has deleted from that memory location while pointer is still pointing such memory location. Such pointer is known as dangling pointer and this problem is known as dangling pointer problem.

Ex:-

```

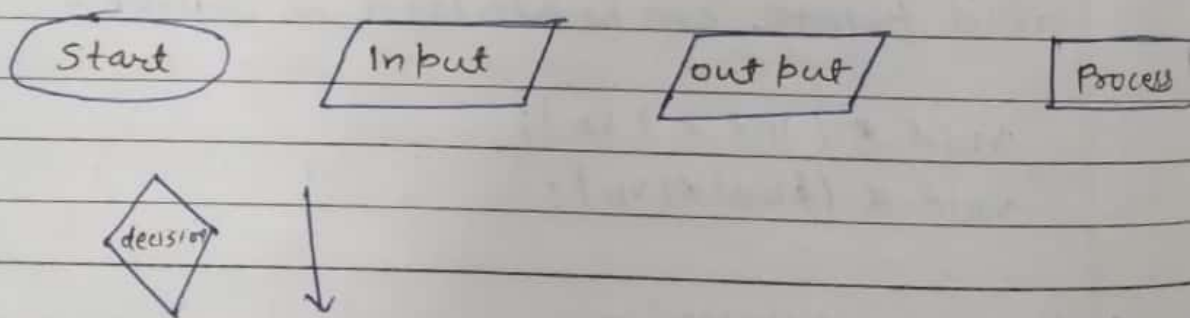
{
    int a;
    a = 5;
    int *p;
    p = &a;
    printf("%d", *p) → 5;
    free(*p);
}
→ printf("%d", *p);
    
```

28/10/23

Algorithm & flow chart :-

Algorithm → step by step process.

flow chart → graphical representation.

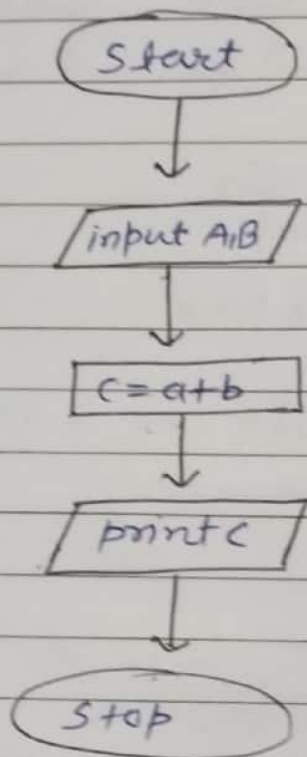


Algorithm:-

Step 1	Start
Step 2	input A, B
Step 3	$C = A + B$
Step 4	print C
Step 5	stop

flow chart:-

9

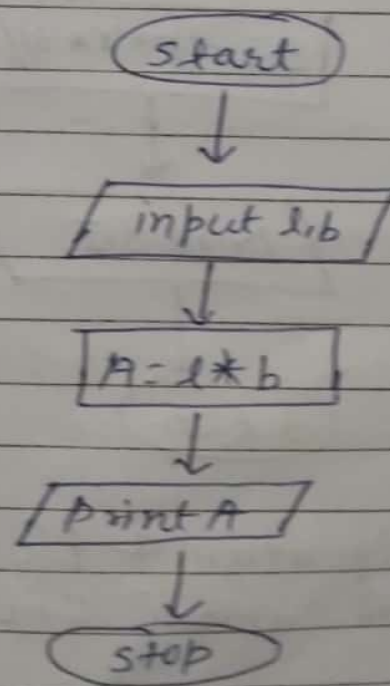


* Algorithm & flow chart of Area of rectangle.

Algorithm:-

- Step 1:- Start
- Step 2:- input l, b
- Step 3:- $A = l * b$
- Step 4:- print A
- Step 5:- stop

flowchart:-



Q. Algorithm & flowchart of simple interest.

10

Algorithm -

Step 1:- Start

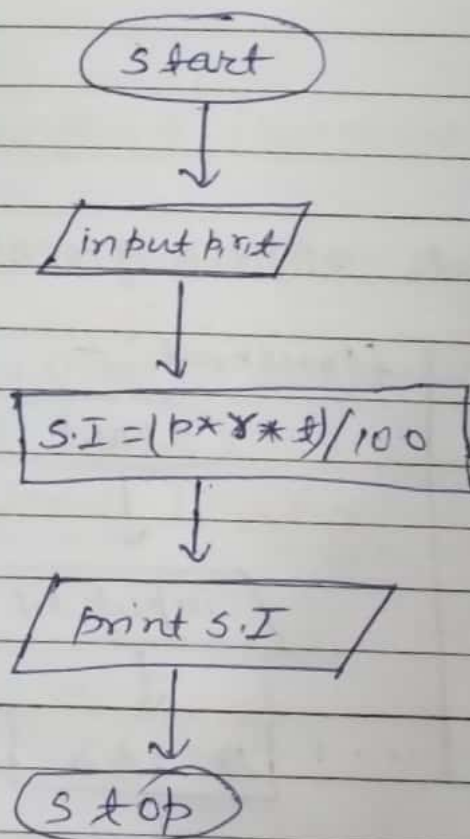
Step 2 - input p, r, t

Step 3 $SI = (p \times r \times t) / 100$

Step 4 Print $S.I$

Step 5 Stop

Flowchart -



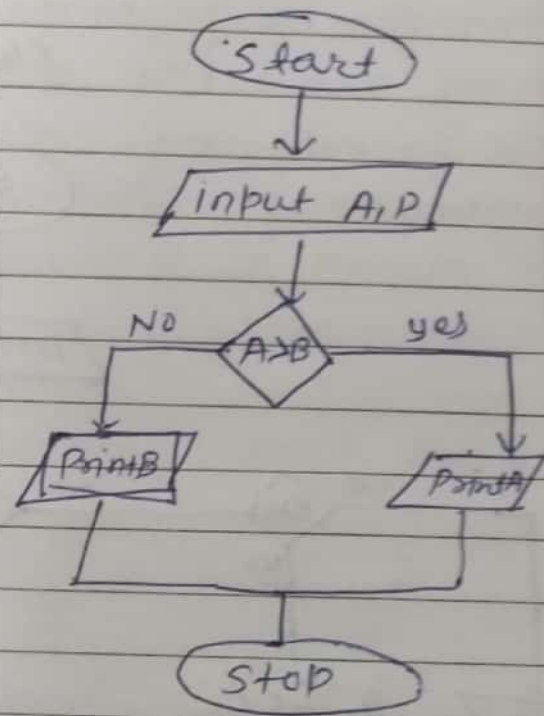
Q. Algorithm & flowchart of to print largest no. among 2 no's.

Algorithm:-

Step 1 Start
 Step 2 input A, B
 Step 3 if $A > B$
 print A
 else
 print B
 end if

Step 4 — Stop

flow chart



Q Algorithm and flowchart to print largest no. among 3 no's.

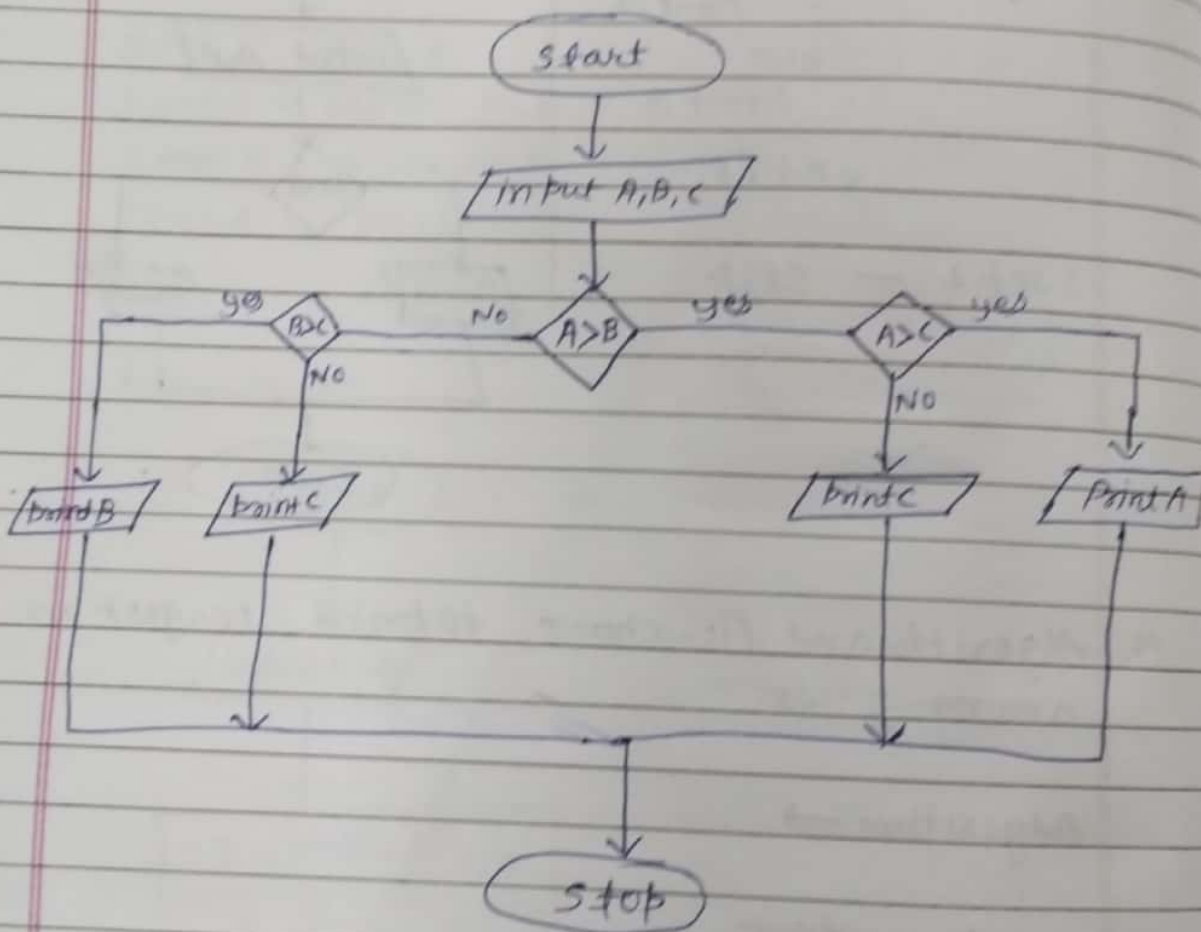
Algorithm →

Step 1- start
 Step 2- input A, B, C
 Step 3: if $A > B$
 if $A > C$
 print A
 else
 print C
 end if
 else
 if $B > C$
 print B
 else
 print C
 end if
 end if

step 4 stop

12

flow chart



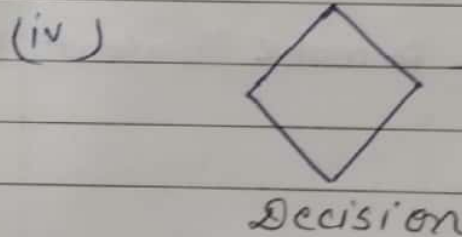
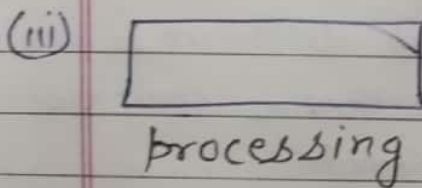
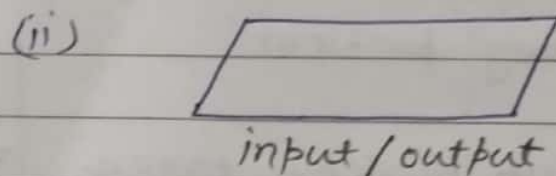
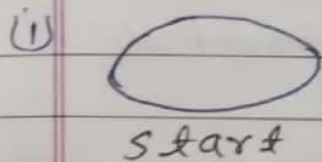
Homework:- 28/10/23

Q. Define Algorithm and flowchart difference b/w Algorithm and flowchart write an algorithm and flowchart of largest No. amongs three Numbers

Algorithm:- An algorithm is a procedure or step by step instruction for solving a problem. They form the foundation of writing a program.

Flowchart:- Flowchart is a graphical representation of an algorithm. It makes use of symbols which are connected among them to indicate the flow of information and processing. The process of drawing a flowchart for an algorithm is known as "flowcharting".

Basic symbols used in flow chart Designs:-



(v)  = Arrows represent the direction of flow of control and relation

Difference b/w Algorithm & flow chart:-

Algorithm	flow chart
1. Step by step instruction representing the process of any solution.	1. Block by block information diagram representing the data flow.
2. It is step wise analysis of the work to be done.	2. It is a pictorial representation of a process.

- | | |
|--|---|
| 3. Solution is shown in non-computer language like english | 3. solution is shown in graphical format |
| 4. It is somewhat difficult to understand | 4. Easy to understand as compared to algorithm. |
| 5. Difficult to show branching and looping. | 5. Easy to show branching and looping. |
| 6. Algorithm can be written for any problem | 6. Flowchart for big problem is impractical. |
| 7. Difficult to debug errors. | 7. Easy to debug errors. |
| 8. It is easy to make flowchart | 8. It is difficult to write algorithm as compared to flowchart. |

* algorithm and flowchart to find largest no. among 3 no.

Step 1: Start

Step 2: input A, B, C

Step 3: if (A > B)

 if (A > C)

 print A

 else

 print B

 end if

else

 if (B > C)

 print B

 else

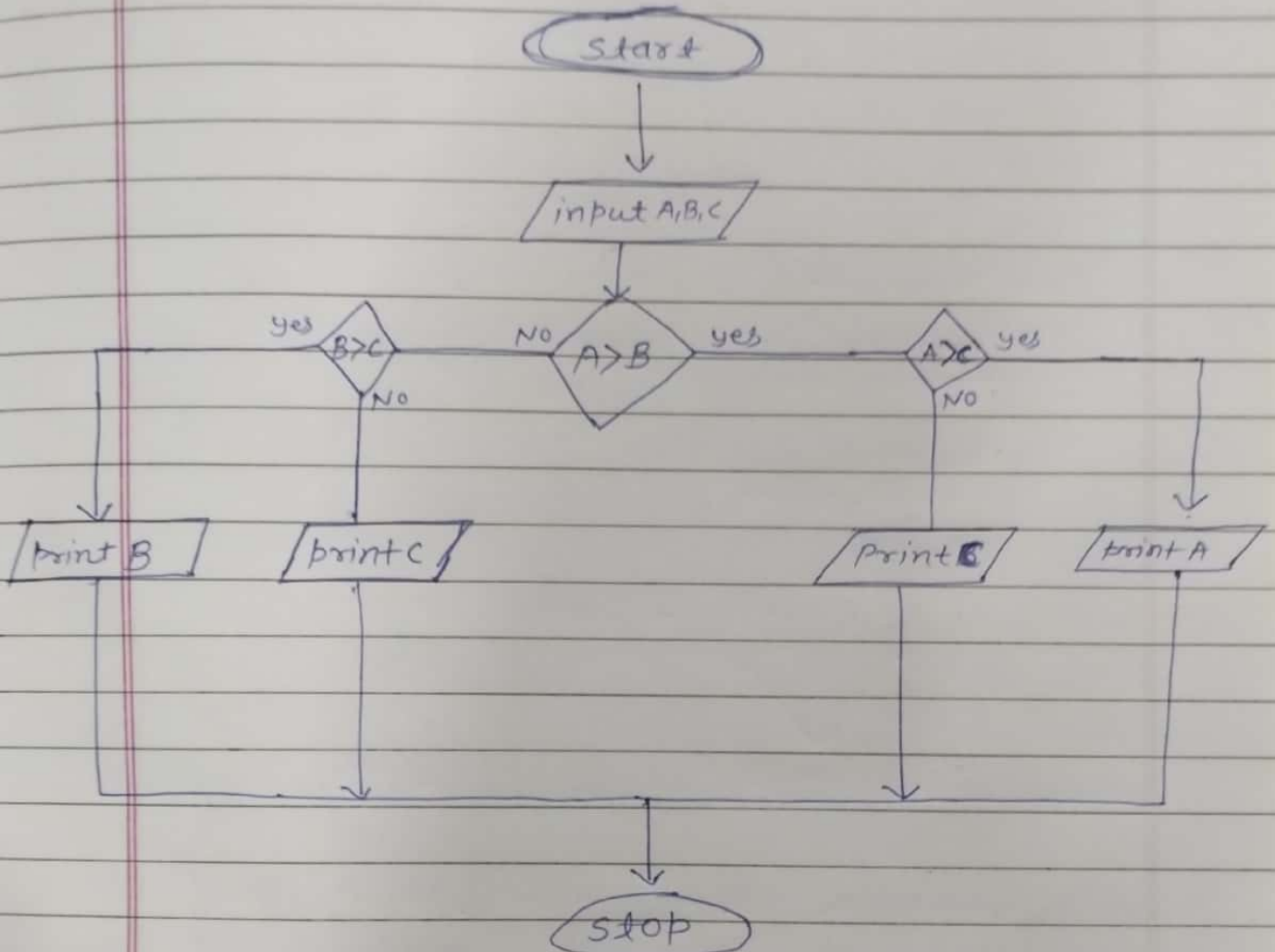
 print C

 end if

end if

step 4: stop

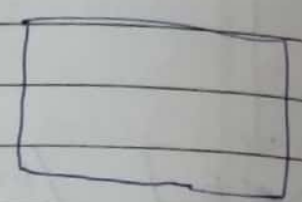
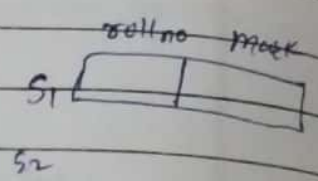
Flow chart



Structure → user defined datatype.
 ↳ stores different data types

Syntax

```
struct    structure Name.
{
    data type 1;
    data type 2;
    data type 3;
};
```



Sample Program

```
#include <stdio.h>
#include <conio.h>
```

```
struct student
{
    int roll no;
    float marks;
};
```

```
void main ()
{
    struct student s1, s2;
    clrscr();
    printf("enter roll no. of s1 & s2");
    scanf("%d %d", &s1.rollno, &s2.rollno);
    printf("enter marks of s1 & s2");
    scanf("%f %f", &s1.marks, &s2.marks);
    printf("The roll no & marks of s1 are as %d %f",
        s1.rollno, s1.marks);
}
```

```
printf("The rollno & marks of s2 are as %d %f", s2.
rollno, s2.marks);
getch();
}
```

Q. Write a C program to print students name two students which have more marks by using structure.

```
#include <stdio.h>
#include <conio.h>
// #include <string.h>
struct student
{
    char student name[10];
    float marks;
};

void main()
{
    struct student s1, s2;
    clrscr();
    printf("enter the name of student s1 & s2");
    scanf("%s %s", s1.name, s2.name);
    printf("enter the marks of the student s1 & s2");
    scanf("%f %f", &s1.marks, &s2.marks);
    printf("The name & marks
    if (s1.marks > s2.marks)
    {
        printf("%s", s1.name);
    }
    else
    {
        printf("%s", s2.name);
    }
    getch();
}
```

Array of structure:-

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
struct student
```

```
{
```

```
int x;
```

```
float m;
```

```
};
```

```
void main ()
```

```
{
```

```
int i;
```

```
struct student A[5];
```

```
clrscr();
```

```
for (i=0; i<5; i++)
```

```
{
```

```
printf("enter roll no");
```

```
scanf("%d", &A[i].x);
```

```
printf("enter marks");
```

```
scanf("%d", &A[i].m);
```

```
}
```

```
printf("The roll no & marks of a student are as");
```

```
for (i=0; i<5; i++)
```

```
{
```

```
printf("%d %f", A[i].x, A[i].m)
```

```
}
```

```
getch();
```

```
}
```

✓✓✓

write a c-program to print highest the students name which have highest marks among various students by using structure.

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
struct student
{
    char n[10];
    float m;
};
void main()
{
    struct student A[5];
    int i, max, k;
    clrscr();
    for(i=0; i<5; i++)
    {
        printf("enter name");
        scanf("%s", A[i].n);
        printf("enter marks");
        scanf("%f", &A[i].m);
    }
    max = A[0].m;
    for(i=1; i<5; i++)
    {
        if(max < A[i].m)
        {
            max = A[i].m;
            k = i;
        }
    }
}
```

printf("%s", A[k].n);

getch();

}

- Q. print an employee name detail by using structure whose salary is greater than 20K & experience is more than 5 years.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#include <string.h>
```

```
struct emplog
```

```
{
```

```
    char n[10];
```

```
    int sl;
```

```
    int exp;
```

```
}
```

```
void main()
```

```
{
```

```
    int i, max;
```

```
    clrscr();
```

```
    struct employee e[5];
```

```
    for(i=0; i<5; i++)
```

```
    {
```

```
        printf("enter employee name");
```

```
        scanf("%s", e[i].n);
```

```
        printf("enter employee salary");
```

```
        scanf("%d", &e[i].sl);
```

```
        printf("enter experience");
```

```
        scanf("%d", &e[i].exp);
```

PAGE No.	
DATE	

```

}
for (i=0; i<5; i++)
{
    if (e[i].sal > 20000 & e[i].exp > 5)
    {
        printf("The employee name is, " e[i].n)
    }
}
}
getch();
}

```

Q. Print cricketer Jersey no. , cricketer name whose total runs in a career is more than 10000 but less than 20000.

```

#include <stdio.h>
#include <conio.h>
#include <string.h>
struct cricketer
{
    char n[10];
    int j;
    int R;
};
void main()
{
    int i;
    clrscr();
    struct cricketer c[5];
    for (i=0; i<5; i++)

```

```

{
    printf("enter cricketer name");
    scanf("%s", c[i].n);

    printf("enter cricketer jersey no.");
    scanf("%d", &c[i].j);
    printf("enter cricketer run");
    scanf("%d", &c[i].R);

    for(i=0; i<5; i++)
    {
        if(R > 10000 & & R < 20000)
        {
            printf("The name of cricketer & jersey no. is  

            %s, %d", c[i].n, c[i].j);
        }
    }
    getch();
}

```

Jump statements :-

- (i) Break (ii) continue
- (iii) goto (iv) return

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i;
    clrscr();
    for (i=0; i<10; i++)
    {
        if (i==3 || i==7)
        {
            continue;
        }
        printf("%d", i);
        getch();
    }
}
```

goto ⇒

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int s1, s2, s3, s4, s5, T, Avg;
    clrscr();
    printf("enter marks of subjects out of 50");
    scanf("%d %d %d %d %d", &s1, &s2, &s3, &s4, &s5);
    T = s1 + s2 + s3 + s4 + s5;
    goto l2;
```

```

    Avg = T/5 ; } skip
    t1;

```

```

printf ( "Total marks is %d", T );
printf ( "The average is %d", avg );

```

Syntax of do-while

```

initialization
do
{

```

```

    increment
} condition;

```

# while loop - (Entry control)	Do-while () (Exit control)
void main ()	void main ()
{	{
int i ;	int i ;
clrscr ()	clrscr () ;
i = 10 ;	i = 10
while (i == 4)	do
{	{
printf (" Jbit ") ;	printf (" Jbit ") ;
i++ ;	i++ ;
}	} while (i == 4) ;
getch () ;	getch () ;
}	}

File system or file handling

File $\xrightarrow{\text{store information or data}}$ manage

W. docx or doc

W. xls

W. ppt

W. exe

W. C

etc.

1. read  A

2. write  B

3. Delete

4. Append

Define file. explain its ^{basic} operations. write down the characteristic of file handling.

File:- A file is a collection of logically & related data that is recorded ~~data~~ on the secondary storage in the form of sequence of operations.

The content of the files are defined by its creator who is creating the file.

* Its basic operations:-

(i) Read (ii) write (iii) Delete (iv) Append

(i) Read- This operation reads the contents from a file. A Read pointer is maintained by the OS. pointing to the position up to which the data has been read.

(ii) write operation:-

This operation is used to write the information into a file. A system call write is issued that specifies the name of the file and the length of the data has to be written to the file. Whenever the file length is increased by specified value and the file pointer is repositioned after the last byte written.

(iii) Delete operation :-

Deleting the file will not only delete all the data stored inside the file it is also used so that disk space occupied by it is freed. In order to delete the specified file the directory is searched. When the directory entry is located, all the associated file space and the directory entry is released.

(iv) Append operation :-

This operation adds data to the end of the file.

Q. Explain functions of file handling. Describe its modes.

PAGE No.	
DATE	

The various function of file handling are as:-

- (i) `fopen()` - It is used create a file or open a file
- (ii) `fclose()` - It is used to closed a file.
- (iii) `fgets()` - It is used to read a file.
- (iv) `fprintf()` - It is used to write blocks of data into a file.
- (v) `fscanf()` - It is used to read blocks of data ~~int~~ from a file.
- (vi) `getc()` - It is used to read a single character to a file.
- (vii) `putc()` - It is used to write a single character to a file.
- (viii) `fseek()` - It is used to set the position of a file pointer a mentioned location.
- (ix) `ftell()` - It is used to return the current position of a file pointer.

10. `rewind()` - It is used to set the file pointer to the beginning of the file.

11. `putw()` - It is used to write an integer to a file.

12. `getw()` - It is used to read an integer from a file.

13. The various modes of file handling:-

- (i) `r` → open for reading.
- (ii) `rb` → open for reading in binary mode.
- (iii) `w` → open for writing.
- (iv) `wb` → open for writing in binary mode.
- (v) `a` → open for Append. data that is to be added to the file.
- (vi) `ab` → open for append to the file data is added to the file.
- (vii) `r+` → open for both reading & writing.
- (viii) `rb+` → open for both reading & writing in binary mode.
- (ix) `w+` → open for both reading & writing.
- (x) `wb+` → open for both reading & writing in binary mode.
- (xi) `a+` → open for reading & append.
- (xii) `ab+` → open for both reading & appending in binary mode.